TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS • BACK-END

HTTP, REST API e Express

Como cliente e servidor conversam — e como construir o seu próprio servidor

```
app.get('/', (req, res) ⇒ {
  res.send('Olá, turma!');
});
```

PROFESSOR

Tarik Ponciano

BACK-END COM NODE.JS

Roteiro

01 **HTTP**
Como cliente e servidor conversam pela web

02 **REST API**
Recursos, rotas e métodos do jeito certo

03 **Express**
O que é e a configuração inicial do projeto

04 **Primeiro servidor**
Rotas GET, respostas em JSON e middleware

05 **Thunder Client**
Testar a API a cada rota nova

06 **CRUD no Express**
Listar, buscar, criar, atualizar e remover — com JSON

NESTE MATERIAL

Exemplo explicado
O professor resolve um caso: a API de alunos.

Prática
Você aplica em outro caso: a API da lanchonete.

Código JavaScript
Pronto para copiar no server.js e rodar com node.

ONDE ESTAMOS

Do terminal para a rede

ATÉ AGORA · TERMINAL

Node.js + Inquirer

```
const nome = await input({
  message: 'Qual o seu nome?'
});
```

O programa conversa com **quem está no teclado**, um de cada vez.

HOJE · REDE

Node.js + Express

```
app.get('/alunos', (req, res) ⇒ {
  res.json(alunos);
});
```

O programa responde a **qualquer cliente pela rede**: navegador, app de celular, outro sistema.

O JavaScript é o mesmo. Muda **quem faz a pergunta**: em vez do teclado, chega uma **requisição HTTP**.

SEÇÃO 01

HTTP

01

O protocolo que leva pedidos e respostas entre cliente e servidor.

01 • HTTP

De onde vêm os posts?

Você abre o app e o feed aparece em menos de um segundo. **Aqueles posts não estavam no seu celular.**

1. **O app pede** os posts a um computador distante: o **servidor**.
2. **O servidor busca** os dados e manda de volta.
3. **O app desenha** a resposta na tela.

Aquecimento: quantos pedidos desse tipo o seu celular já fez hoje, desde que você acordou?

POR TRÁS DE CADA TOQUE

Abrir o feed
pede a lista de posts

Enviar uma mensagem
manda um texto novo para guardar

Fazer um Pix
registra um pagamento

Digitar o CEP na loja
pede o endereço daquele CEP

01 • HTTP

Cliente pergunta, servidor responde

Cliente

Navegador, app de celular ou o Thunder Client.

Sempre começa a conversa.

REQUISIÇÃO

GET /alunos

RESPOSTA

200 OK + lista em JSON

Servidor

Um programa que fica esperando pedidos. Hoje: o seu, com Express.

Nunca puxa assunto.

ANALOGIA DO RESTAURANTE

Cliente
quem senta à mesa e faz o pedido

Requisição
o pedido anotado e levado à cozinha

Resposta
o prato que volta, com um aviso de como foi

01 • HTTP • ANATOMIA

Anatomia de uma requisição

```
POST /alunos HTTP/1.1                          ← método e caminho
Host: localhost:3000                           ← para quem
Content-Type: application/json                 ← formato do corpo

{ "nome": "Ana", "turma": "TDS-01" }           ← corpo
```

AS PEÇAS

Método
O que fazer: GET, POST, PUT, DELETE.

Caminho (URL)
Sobre o quê: /alunos.

Headers
Informações extras, como o formato do corpo.

Corpo (body)
Os dados enviados, em JSON. GET não tem corpo.

01 · HTTP · ANATOMIA

Anatomia de uma resposta

```
HTTP/1.1 201 Created                          ← status: como foi
Content-Type: application/json                ← formato

{ "id": 3, "nome": "Ana", "turma": "TDS-01" }  ← corpo
```

AS PEÇAS

Status

Um número que resume o resultado.

Headers

Informações extras, como o formato da resposta.

Corpo (body)

Os dados devolvidos, quase sempre em JSON.

Toda requisição recebe uma resposta, **mesmo quando dá erro**. É o **status** que conta o que aconteceu.

01 • HTTP

Os métodos: o verbo do pedido

GET

Buscar

Lê dados sem mudar nada. É o que o navegador faz ao abrir um endereço.

```
GET /alunos
```

POST

Criar

Envia dados novos no corpo para o servidor guardar.

```
POST /alunos
```

PUT

Atualizar

Troca os dados de algo que já existe.

```
PUT /alunos/3
```

DELETE

Remover

Apaga algo que já existe.

```
DELETE /alunos/3
```

Existem outros métodos, como PATCH e OPTIONS. Para o **CRUD** de hoje — **C**riar, **R**ead (ler), **U**pdate (atualizar), **D**elete — estes quatro bastam.

01 • HTTP

Códigos de status: como foi?

2xx **Sucesso**
o pedido foi atendido

4xx **Erro do cliente**
o pedido veio errado

5xx **Erro do servidor**
o código do back-end quebrou

Código	Significa	Quando aparece hoje
200 OK	Deu certo	Listar, buscar ou atualizar
201 Created	Algo foi criado	Depois de um POST
204 No Content	Deu certo, sem corpo	Depois de um DELETE
400 Bad Request	Pedido incompleto	Faltou um campo obrigatório
404 Not Found	Não existe	Buscar um id que não existe
500 Internal Server Error	Erro no servidor	Bug no código do back-end

Para memorizar: **2xx** deu certo, **4xx** a culpa é de quem pediu, **5xx** a culpa é do servidor.

01 · HTTP

JSON: o idioma dos dados

JSON — *JavaScript Object Notation* — é **texto** no formato de um objeto. É assim que cliente e servidor trocam dados, seja qual for a linguagem de cada um.

JAVASCRIPT · OBJETO NO CÓDIGO

```
const aluno = {
  nome: 'Ana',
  turma: 'TDS-01',
  ativo: true
};
```

JSON · O QUE VIAJA PELA REDE

```
{
  "nome": "Ana",
  "turma": "TDS-01",
  "ativo": true
}
```

Chaves: sempre entre aspas duplas

Valores: texto, número, true/false, null, listas

Sem vírgula sobrando no final

No Express, **res.json(objeto)** transforma o objeto em JSON para enviar, e **express.json()** faz o caminho de volta ao receber.

PRÁTICA 1 • 10 MIN • INDIVIDUAL

Espie as requisições

No navegador, aperte **F12** e abra a aba **Network** (Rede). Com ela aberta:

a Abra um site que você usa todo dia e recarregue a página.

b Clique em três requisições e anote o **método** e o **status** de cada uma.

c Acesse viacep.com.br/ws/SEU_CEP/json/ com o seu CEP e olhe a resposta.

d Troque o CEP por 6000 (só 4 dígitos) e compare o status.

PARA CADA UMA

```
Método: …
Caminho: …
Status: …
Formato: …
```

O formato aparece no header **Content-Type** da resposta.

PENSE E RESPONDA Por que o CEP de 4 dígitos não volta 200? De quem é a culpa: do cliente ou do servidor?

SEÇÃO 02

REST API

02

Um jeito combinado de desenhar rotas: a URL diz o quê, o método diz o que fazer.

02 • REST

API e REST

API

Application Programming Interface

O **cardápio de pedidos** que um sistema aceita: o que dá para pedir e como pedir.

REST

Representational State Transfer

Um **estilo de desenhar APIs** sobre HTTP: cada coisa do sistema tem um endereço, e o método diz a ação.

UMA API REST BEM FEITA TEM

Recursos
Substantivos no plural: /alunos, /produtos.

Métodos com sentido
GET lê, POST cria, PUT atualiza, DELETE remove.

Sem memória
Cada requisição traz tudo o que precisa.

JSON
Dados entram e saem no mesmo formato.

REST não é biblioteca nem linguagem: é uma **convenção**. O Express é a ferramenta; o REST é o jeito de organizar as rotas.

02 · REST

Um recurso, cinco rotas

O recurso **alunos** vira o endereço /alunos. Um aluno específico é encontrado pelo id: /alunos/3.

Ação	Método	Rota	Corpo enviado	Sucesso
Listar todos	GET	/alunos	nenhum	200
Buscar um	GET	/alunos/:id	nenhum	200 (ou 404)
Criar	POST	/alunos	JSON do novo aluno	201 (ou 400)
Atualizar	PUT	/alunos/:id	JSON com os novos dados	200 (ou 404)
Remover	DELETE	/alunos/:id	nenhum	204 (ou 404)

:id é um **parâmetro de rota**: um espaço reservado. Em /alunos/3, o id vale 3.

02 · REST

Rota boa, rota ruim

EVITE	PREFIRA	POR QUÊ
`GET /criarAluno`	`POST /alunos`	A ação já está no método.
`POST /alunos/deletar/3`	`DELETE /alunos/3`	Use o método certo para a ação.
`GET /aluno e GET /alunos`	`GET /alunos`	Escolha o plural e mantenha.
`PUT /atualizar?id=3`	`PUT /alunos/3`	O id faz parte do endereço do recurso.

Teste rápido: se dá para **ler a rota em voz alta sem verbo** e o método completa a frase, ela está no padrão REST.

PRÁTICA 2 • 10 MIN • DUPLAS

Desenhe as rotas da lanchonete

A lanchonete da escola quer uma API para o cardápio — é ela que você vai programar nas próximas práticas. **Por enquanto, sem código**: escreva o método e a rota de cada pedido.

- **a** Ver todo o cardápio.
- **b** Ver só o produto de id 4.
- **c** Cadastrar o novo suco de cajá.
- **d** Mudar o preço do salgado de id 2.
- **e** Tirar do cardápio o produto de id 7.
- **f** **Desafio:** como pedir só as bebidas?

FORMATO DA RESPOSTA

```
a) GET /produtos
b) …
```

Use o recurso **produtos**, no plural, em todas as rotas.

PENSE E RESPONDA Quais desses pedidos precisam de um corpo em JSON? O que vai dentro dele?

SEÇÃO 03

Express

03

O framework que transforma o Node.js em servidor web com poucas linhas — e como preparar o projeto.

03 · EXPRESS

O que é o Express

Express é um **framework** para Node.js que facilita criar servidores web e APIs.

- É um pacote do npm, instalado em cada projeto.
- Cuida dos detalhes do HTTP. Você só diz: **“nesta rota, faça isto”**.
- Minimalista: começa pequeno, e o resto entra quando você precisa.

Daria para fazer tudo só com o Node.js puro, mas com muito mais código para cada rota.

UMA ROTA

```
app.get('/alunos', (req, res) ⇒ {
  res.json(alunos);
});
```

AS PEÇAS

`app`	o servidor
`.get`	o método HTTP que esta rota atende
`'/alunos'`	o caminho
`(req, res)`	o pedido que chegou e a resposta que vai sair
`res.json()`	envia a resposta em JSON

Configuração inicial

1 **Crie a pasta do projeto e entre nela**

```
mkdir api-alunos
cd api-alunos
```

2 **Crie o package.json e ative o import**

```
npm init -y
npm pkg set type=module
```

3 **Instale o Express** • baixa para node_modules

```
npm install express
```

4 **Abra no VS Code e crie o arquivo server.js**

```
code .
```

COMO FICA A PASTA

```
api-alunos/
├── node_modules/          pacotes
├── package.json           ficha do projeto
├── package-lock.json
└── server.js              o seu código
```

⚠ **node_modules** não vai para o GitHub: crie um **.gitignore** com essa pasta. Quem baixar o projeto roda **npm install**.

03 · EXPRESS · CONFIGURAÇÃO

import ou require?

PACKAGE.JSON · DEPOIS DO PASSO 2

```
{
  "name": "api-alunos",
  "main": "index.js",
  "type": "module",                    ← libera o import
  "dependencies": { … }
}
```

IMPORT · PADRÃO DA TURMA

```
import express from 'express';
```

REQUIRE · JEITO ANTIGO

```
const express = require('express');
```

	import (ES Modules)	require (CommonJS)
Precisa de	"type": "module" no package.json	nada: é o padrão antigo do Node
Onde aparece	JavaScript moderno, front-end, React	projetos e tutoriais mais antigos
Arquivo seu	import { x } from './dados.js'	require('./dados')
Atenção	a extensão .js é obrigatória	não mistura com import no mesmo arquivo

SEÇÃO 04

Primeiro servidor

04

Rotas GET, respostas em JSON e o primeiro middleware.

Anatomia de uma rota

```
app.get('/alunos/:id', (req, res) => {
  res.json(aluno);
});
```

1 2 3 4 5

1 • O servidor
O app criado com express().

2 • O método
get, post, put ou delete: o verbo que a rota atende.

3 • O caminho
Entre aspas. :id marca um parâmetro, lido em req.params.id.

4 • A função
Roda quando método e caminho combinam com o pedido.

5 • A resposta
Uma por pedido: res.send, res.json ou res.status(...).

O MOLDE `app.método(caminho, (req, res) => { resposta });`

04 · PRIMEIRO SERVIDOR · EXEMPLO EXPLICADO

server.js: o servidor mínimo

```
import express from 'express';                          ← traz o Express (precisa do "type": "module")
const app = express();                                  ← cria o servidor

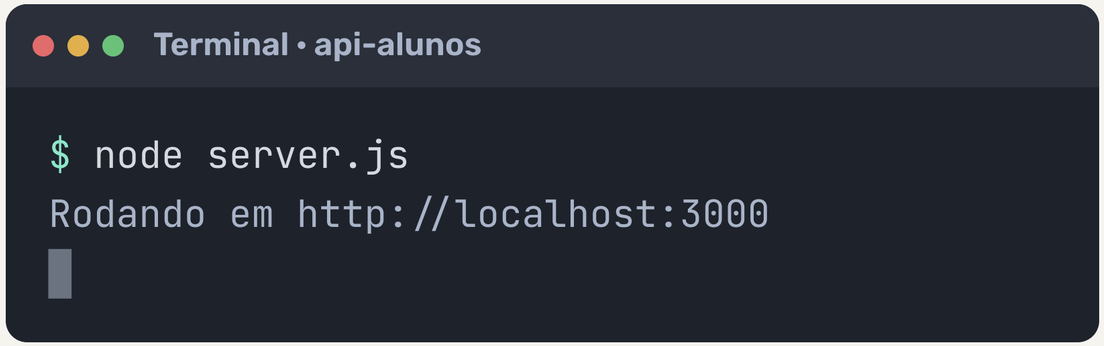
app.get('/', (req, res) => {                            ← rota GET no caminho /
  res.send('Olá, turma! Servidor no ar.');              ← a resposta
});

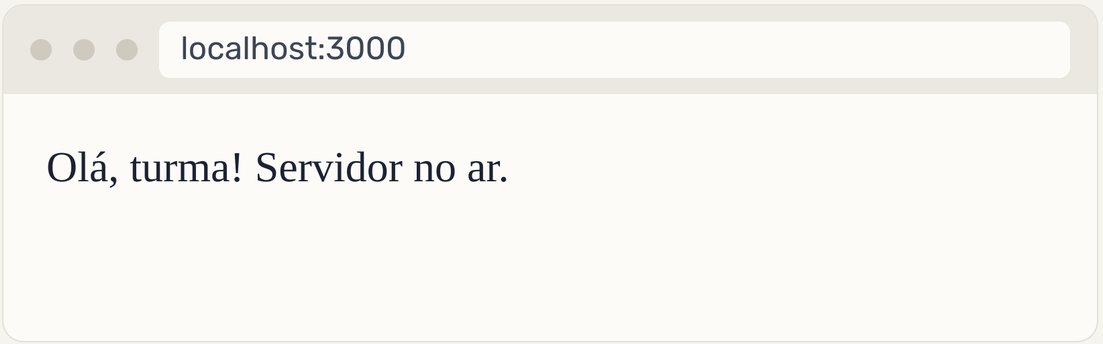
app.listen(3000, () => {                                ← fica ouvindo na porta 3000
  console.log('Rodando em http://localhost:3000');      ← aviso no terminal
});
```

Salve, rode **node server.js** e abra **http://localhost:3000** no navegador: a mensagem é a resposta da sua rota.

04 • PRIMEIRO SERVIDOR

Rodando o servidor





O terminal “trava”
É normal: o servidor está esperando pedidos.

Ctrl + C para parar
Desliga o servidor e libera a porta 3000.

node --watch server.js
Reinicia sozinho cada vez que você salva o arquivo.

04 • PRIMEIRO SERVIDOR

req e res: o pedido e a resposta

REQ • O QUE CHEGOU

`req.params`

Pedaços da rota. Em /alunos/:id → **req.params.id**

`req.query`

O que vem depois do ?. Em /produtos?categoria=bebida → **req.query.categoria**

`req.body`

O corpo em JSON. Só funciona com **app.use(express.json())**

RES • O QUE VAI VOLTAR

`res.send('texto')`

Envia um texto.

`res.json(objeto)`

Envia um objeto ou uma lista em JSON.

`res.status(201).json(novo)`

Escolhe o status e envia, em uma linha só.

Cada rota responde **uma única vez** — e precisa responder, senão o cliente fica esperando.

04 · PRIMEIRO SERVIDOR · EXEMPLO EXPLICADO

Uma rota que devolve JSON

SERVER.JS · ABAIXO DA ROTA /

```
app.get('/sobre', (req, res) => {
  res.json({
    sistema: 'API de Alunos',
    curso: 'Desenvolvimento de Sistemas',
    versao: 1
  });
});
```

localhost:3000/sobre

```
{
  "sistema": "API de Alunos",
  "curso": "Desenvolvimento de Sistemas",
  "versao": 1
}
```

res.json em vez de res.send

O objeto vira JSON e a resposta já sai com Content-Type: application/json.

Rota nova = um app.get novo

O servidor pode ter quantas rotas precisar.

PRÁTICA 3 • 15 MIN • INDIVIDUAL

Seu servidor: a lanchonete

O professor fez a **API de alunos**. Você faz a **API da lanchonete**, que vai crescer até o fim da aula.

a Crie o projeto **api-lanchonete** com a configuração inicial e o "type": "module".

b **GET /** responde com o texto “Bem-vindo à lanchonete!”.

c **GET /sobre** responde em JSON com o nome, o horário e as formas de pagamento.

d Acesse **/xyz**, que não existe. Qual status o F12 mostra?

ESPERADO EM /sobre

```
{
  "lanchonete": "Cantina da Escola",
  "horario": "18h às 22h",
  "pagamento": ["dinheiro", "pix"]
}
```

Modelo: as rotas **/** e **/sobre** da API de alunos.

PENSE E RESPONDA Quem escreveu a página de erro do /xyz? Você não criou essa rota.

Middleware: o meio do caminho

Middleware é uma função que roda **entre a chegada da requisição e a rota**. Ela pode olhar o pedido, preparar dados para a rota ou barrar o pedido.



Como a portaria de um prédio

Todo visitante passa pela portaria antes de chegar ao apartamento. Ela anota quem entrou, confere o que a pessoa traz e pode barrar a entrada.

E o app.use?

app.use(função) registra um middleware que vale para **todas** as requisições, de qualquer método e caminho. Ele roda na ordem em que foi escrito: por isso vem **antes** das rotas.

Anatomia de um middleware

```
app.use((req, res, next) ⇒ {
  console.log(req.method, req.url);

  next();
});
```

1 • app.use
Registra o middleware. Sem caminho: vale para toda requisição.

2 • (req, res, next)
Os mesmos req e res da rota, mais o next.

3 • O trabalho
Olhar, preparar ou barrar o pedido. Aqui: registrar no terminal.

4 • next()
Passa o pedido para o próximo middleware ou para a rota.

	Rota	Middleware
Declaração	app.get('/alunos', função)	app.use(função)
Quando roda	Só se método e caminho combinam	Em toda requisição, antes das rotas
Termina com	res.json(...): responde ao cliente	next(): passa adiante (ou responde, se barrar)

04 · PRIMEIRO SERVIDOR · EXEMPLO EXPLICADO

app.use na prática

```
import express from 'express';
const app = express();

// 1. registra cada pedido no terminal
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

// 2. lê o JSON do corpo e preenche req.body
app.use(express.json());

// 3. as rotas vêm depois
app.get('/', (req, res) => { ... });
```

Terminal · api-alunos

```
$ node --watch server.js
Rodando em http://localhost:3000
GET /
GET /sobre
```

Pelo navegador pode aparecer também GET /favicon.ico: é o navegador pedindo o ícone da aba.

next()

Passa o pedido adiante. Sem ele, o pedido para ali e o cliente fica esperando.

express.json()

Middleware que já vem pronto. Sem ele, req.body chega undefined.

SEÇÃO 05

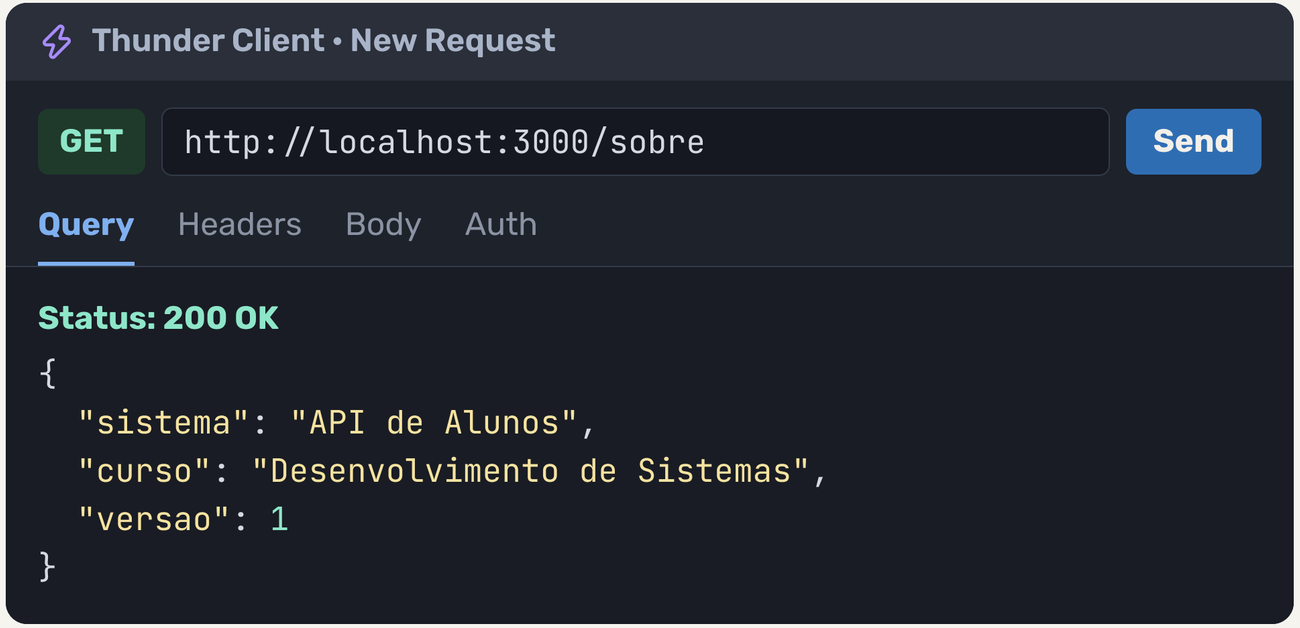
Thunder Client

05

Testar a API sem escrever um front-end — a cada rota nova.

05 · THUNDER CLIENT

Um cliente HTTP dentro do VS Code



COMO COMEÇAR

1. Extensões (**Ctrl+Shift+X**): busque **Thunder Client** e instale.
2. Clique no ícone de raio na barra lateral e em **New Request**.
3. Escolha o **método** e digite a **URL**.
4. Vai enviar dados (POST ou PUT)? Aba **Body** → **JSON**.
5. **Send** e leia o **status** e a resposta.

Sem Thunder Client? O Postman faz o mesmo.

05 • THUNDER CLIENT

Teste a cada rota nova

1

Escreva a rota

Uma rota por vez, no server.js.

→

2

Salve o arquivo

Com node --watch, o servidor reinicia sozinho.

→

3

Envie pelo Thunder

Mesmo método e mesma URL da rota.

→

4

Confira o resultado

Status, corpo da resposta e terminal.

Não deu o esperado? Leia a resposta e o terminal, corrija e volte ao passo 1.

Guarde numa coleção

Salve cada requisição numa coleção (aba Collections). Para repetir um teste, é só abrir e clicar em Send.

Olhe o terminal

O middleware de registro mostra cada pedido. Se a linha não aparece, o pedido nem chegou ao servidor.

PRÁTICA 4 • 15 MIN • INDIVIDUAL

Middleware e testes na lanchonete

Leve para a **api-lanchonete** o que o professor fez na API de alunos.

- **a** Adicione o middleware de registro e o **express.json()**, antes das rotas.
- **b** Instale o Thunder Client e crie a coleção **Lanchonete**.
- **c** Teste **GET /** e **GET /sobre**: confira o status 200 e o terminal.
- **d** Teste **GET /cardapio**, que ainda não existe. Qual status voltou?

NO TERMINAL

```
GET /
GET /sobre
GET /cardapio
```

Uma linha para cada Send. Até o 404 aparece: o middleware roda antes de qualquer rota.

PENSE E RESPONDA Apague o next() do middleware e teste de novo. Por que o Thunder Client fica esperando?

SEÇÃO 06

CRUD no Express

06

O professor monta o CRUD de alunos; você monta o da lanchonete, testando cada rota.

01 • HTTP | 02 • REST | 03 • Express | 04 • Servidor | 05 • Testes | 06 • CRUD

06 · CRUD · EXEMPLO EXPLICADO

Nossos dados: uma lista de objetos

SERVER.JS · DEPOIS DOS MIDDLEWARES

```
let alunos = [
  { id: 1, nome: 'Ana', turma: 'TDS-01' },
  { id: 2, nome: 'Bruno', turma: 'TDS-01' }
];
let proximoId = 3;
```

Um array de objetos

Cada objeto é um aluno, sempre com as mesmas propriedades: id, nome e turma.

Antes das rotas

As rotas usam a lista: ela precisa existir antes delas.

let, e não const

No DELETE, a lista inteira vai ser trocada.

proximoId

Gera o id de cada aluno novo.

⚠ Reiniciou o servidor? A lista volta ao começo: os dados só existem **na memória**. Guardar de verdade é trabalho do banco de dados.

06 • CRUD • READ

Ler: GET /alunos e GET /alunos/:id

LISTAR TODOS

```
app.get('/alunos', (req, res) => {
  res.json(alunos);
});
```

BUSCAR UM PELO ID

```
app.get('/alunos/:id', (req, res) => {
  const id = Number(req.params.id);
  const aluno = alunos.find(a => a.id === id);
  if (!aluno) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }
  res.json(aluno);
});
```

TESTE NO THUNDER CLIENT

GET /alunos	200 • a lista
GET /alunos/1	200 • a Ana
GET /alunos/99	404 • o erro

Por que Number()?

req.params.id chega como texto: '1'. Sem converter, '1' === 1 dá false.

Por que return?

Encerra a rota no erro. Sem ele, o código tentaria responder de novo.

PRÁTICA 5 • 15 MIN • INDIVIDUAL

O cardápio: listar e buscar

PONTO DE PARTIDA • COPIE NO SEU SERVER.JS, ANTES DAS ROTAS

```
let produtos = [
  { id: 1, nome: 'Coxinha', preco: 6.5, categoria: 'salgado' },
  { id: 2, nome: 'Pastel', preco: 7, categoria: 'salgado' },
  { id: 3, nome: 'Refrigerante', preco: 5, categoria: 'bebida' }
];
let proximoId = 4;
```

a

Copie a lista **produtos** e o **proximoId**.

b

GET /produtos devolve o cardápio inteiro.

c

GET /produtos/:id devolve um produto ou **404**.

d

Teste: /produtos/2 dá **200**; /produtos/99 dá **404**.

PENSE E RESPONDA E GET /produtos/dois? Qual status volta, e por quê?

06 · CRUD · CREATE

Criar: POST /alunos

```
app.post('/alunos', (req, res) => {
  const { nome, turma } = req.body;

  if (!nome || !turma) {
    return res.status(400)
      .json({ erro: 'Informe nome e turma' });
  }

  const novo = { id: proximoId++, nome, turma };
  alunos.push(novo);
  res.status(201).json(novo);
});
```

O QUE ACONTECE

1. Pega **nome** e **turma** do corpo.
2. Faltou algum? Responde **400**.
3. Monta o aluno com o próximo id e coloca no fim da lista.
4. Responde **201** com o aluno criado.

const { nome, turma } = req.body

Atalho para pegar duas propriedades do objeto de uma vez.

No Thunder Client: **POST /alunos**, aba Body → JSON. Com nome e turma volta **201**; sem a turma, **400**.

PRÁTICA 6 • 10 MIN • INDIVIDUAL

Novo item no cardápio

a **POST /produtos** pega nome, preco e categoria do corpo.

b Faltou nome ou preço? Responda **400** com uma mensagem de erro.

c Crie o produto com o **proximoId**, coloque na lista e responda **201**.

Modelo: a rota **POST /alunos** do professor.

TESTES NO THUNDER CLIENT

`POST /produtos` **201**

```
{ "nome": "Suco de cajá",
  "preco": 5, "categoria": "bebida" }
```

`POST /produtos` **400**

```
{ "nome": "Tapioca" }
```

`GET /produtos` **200**

O suco aparece no fim da lista, com id 4.

PENSE E RESPONDA Mande "preco": "cinco". A sua API aceitou? Deveria aceitar?

06 · CRUD · UPDATE

Atualizar: PUT /alunos/:id

```
app.put('/alunos/:id', (req, res) => {
  const id = Number(req.params.id);
  const aluno = alunos.find(a => a.id === id);

  if (!aluno) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }

  aluno.nome = req.body.nome ?? aluno.nome;
  aluno.turma = req.body.turma ?? aluno.turma;
  res.json(aluno);
});
```

O QUE ACONTECE

1. Procura o aluno pelo id da rota.
2. Não achou? Responde **404**.
3. Troca só os campos que vieram no corpo.
4. Responde **200** com o aluno atualizado.

E esse ?? aí

novo ?? antigo: usa o valor novo; se ele não veio, mantém o antigo.

O **find** devolve o próprio objeto da lista: mudar **aluno.nome** já muda a lista. Teste: **PUT /alunos/1** com {"turma": "TDS-02"} → **200**.

06 · CRUD · DELETE

Remover: DELETE /alunos/:id

```
app.delete('/alunos/:id', (req, res) => {
  const id = Number(req.params.id);
  const existe = alunos.some(a => a.id === id);

  if (!existe) {
    return res.status(404)
      .json({ erro: 'Aluno não encontrado' });
  }

  alunos = alunos.filter(a => a.id !== id);
  res.status(204).send();
});
```

MÉTODOS DE ARRAY DO CRUD

`find`

Devolve o primeiro que passa no teste.

`some`

true ou false: existe algum que passa?

`filter`

Lista nova só com os que passam. Aqui: todos, menos o removido.

`push`

Coloca um item no fim da lista.

204 No Content: deu certo e não há nada para mostrar. Por isso o **send()** vai vazio. Teste: **DELETE /alunos/2** → 204; repita → **404**.

PRÁTICA 7 • 15 MIN • INDIVIDUAL

Atualizar e tirar do cardápio

a. **PUT /produtos/:id** troca nome, preco ou categoria — só o que veio no corpo.

b. **DELETE /produtos/:id** tira o produto da lista e responde **204**.

c. As duas rotas respondem **404** para um id que não existe.

Modelo: as rotas **PUT** e **DELETE /alunos/:id** do professor.

TESTES NO THUNDER CLIENT

`PUT /produtos/2` — **200**

```
{ "preco": 7.5 }
```

`DELETE /produtos/3` — **204**

`GET /produtos/3` — **404**

`PUT /produtos/99` — **404**

```
{ "preco": 1 }
```

PENSE E RESPONDA Depois do DELETE, o próximo POST recebe qual id? Por que o 3 não é reaproveitado?

06 • CRUD • CONSULTA RÁPIDA

Cola: o CRUD inteiro

Método e rota	O que faz na lista	Sucesso	Erro
GET /alunos	devolve a lista inteira	200	nenhum
GET /alunos/:id	find pelo id	200	404
POST /alunos	push do aluno novo	201	400
PUT /alunos/:id	find e troca os campos	200	404
DELETE /alunos/:id	filter sem o aluno	204	404

REGRAS DE OURO

- **express.json()** antes das rotas
- **Number()** no req.params.id
- **return** antes de responder erro
- uma resposta por requisição
- recurso no **plural**
- salvou? **reinicie** ou use --watch

06 • CRUD • EXEMPLO EXPLICADO

Teste de ponta a ponta: o CRUD de alunos

#	Método e rota	Body (JSON)	Status	Confira
1	GET /alunos	nenhum	200	Ana e Bruno
2	POST /alunos	{"nome":"Carla","turma":"TDS-01"}	201	Carla com id 3
3	POST /alunos	{"nome":"Davi"}	400	faltou a turma
4	GET /alunos/3	nenhum	200	a Carla
5	PUT /alunos/3	{"turma":"TDS-02"}	200	turma nova, mesmo nome
6	DELETE /alunos/3	nenhum	204	resposta vazia
7	GET /alunos/3	nenhum	404	mensagem de erro

Cada teste depende do anterior. O **404 do último** é a prova de que o DELETE funcionou.

PRÁTICA 8 • DESAFIO • 50 MIN • DUPLAS

Uma API nova, do zero

A lanchonete você fez passo a passo. Agora a dupla constrói sozinha a API de outro cliente: projeto, middlewares e **CRUD com JSON**.

Cinema
`/filmes`
titulo, genero, duracao

Biblioteca
`/livros`
titulo, autor, disponivel

Academia
`/clientes`
nome, plano, ativo

Pet shop
`/pets`
nome, especie, tutor

Oficina
`/ordens`
placa, servico, status

Tarefas da turma
`/tarefas`
descricao, concluida

ENTREGAS

1. As cinco rotas do CRUD funcionando.
2. Status certos: 201 ao criar, 400 sem campo obrigatório, 404 para id inexistente.
3. O roteiro de testes do próximo slide, com um print de cada.
+ **Desafio:** filtro com req.query, como /filmes?genero=comedia.

PENSE E RESPONDA Se o servidor reiniciar no meio da apresentação, o que acontece com os dados cadastrados?

PRÁTICA 8 • ROTEIRO DE TESTES

Teste sua API no Thunder Client

Faça na ordem, com o seu recurso. **Antes** de clicar em Send, diga o status que espera.

a	Listar tudo	200	b	Criar um item completo	201
c	Criar sem um campo obrigatório	400	d	Buscar o item criado	200
e	Buscar o id 999	404	f	Atualizar um campo do item	200
g	Remover o item criado	204	h	Buscar o item removido	404

PARA CADA TESTE

```
Esperava: …
Veio: …
Corrigi: …
```

Status diferente do esperado? Leia o **terminal** e o **corpo** da resposta antes de chamar o professor.

PENSE E RESPONDA Qual teste quebrou primeiro na sua API? O que a resposta dizia?

FECHAMENTO

Erros comuns e como resolver

Sintoma	Causa provável	Solução
req.body chega undefined	Faltou express.json() ou o Body não está em JSON	Coloque app.use(express.json()) antes das rotas
EADDRINUSE na porta 3000	Já existe um servidor rodando nela	Ctrl + C no terminal antigo, ou troque a porta
Busca por id sempre dá 404	req.params.id é texto; o id da lista é número	Converta com Number(req.params.id)
Mudei o código e nada mudou	O servidor não foi reiniciado	Reinicie, ou rode com node --watch
Cannot use import statement outside a module	Falta "type": "module" no package.json	Rode npm pkg set type=module
Cannot set headers after they are sent	A rota respondeu duas vezes	Use return antes de cada resposta de erro
Thunder Client fica esperando	Middleware sem next(), ou a rota não respondeu	Chame next() e responda em todo caminho
Thunder Client: ECONNREFUSED	O servidor está desligado	Rode node server.js de novo

FECHAMENTO

Confira antes de chamar o professor

PROJETO E SERVIDOR

- [ ] Express instalado e "type": "module" no package.json.
- [ ] node server.js rodando sem erro no terminal.
- [ ] A rota / responde no navegador.

ROTAS

- [ ] Middlewares (registro e express.json) antes das rotas.
- [ ] Cinco rotas, recurso no plural, sem verbo na URL.
- [ ] return antes de cada resposta de erro.

TESTES

- [ ] Os oito testes feitos no Thunder Client.
- [ ] Status conferidos: 200, 201, 204, 400 e 404.
- [ ] Prints salvos para a entrega.

BACK-END · CONSULTA RÁPIDA

Resumo

HTTP
cliente pergunta
servidor responde
1 pedido, 1 resposta

Métodos
GET lê
POST cria
PUT · DELETE

Status
2xx deu certo
4xx erro do cliente
5xx erro do servidor

JSON
objeto virando texto
chaves com "aspas"
true · false · null

REST
recurso no plural
método diz a ação
/alunos/:id

Express
npm install express
"type": "module"
import express

Rota
app.get(rota, fn)
req.params · body
res.status().json()

Dados
lista de objetos
find · some · filter
zera ao reiniciar

Rodar
node server.js
node --watch
Ctrl + C para

Middleware
app.use(função)
roda antes das rotas
next() passa adiante

Thunder Client
método + URL
Body → JSON
Send e leia

Armadilhas
esqueceu json()
id como texto
sem return

BACK-END • AULA CONCLUÍDA

Sua API está no ar!

Termine o CRUD do seu cenário e os testes no Thunder Client.

PARA ESTUDAR

expressjs.com/pt-br — documentação do Express

developer.mozilla.org — guia de HTTP da MDN

thunderclient.com — documentação do Thunder Client

nodejs.org — documentação do Node.js